

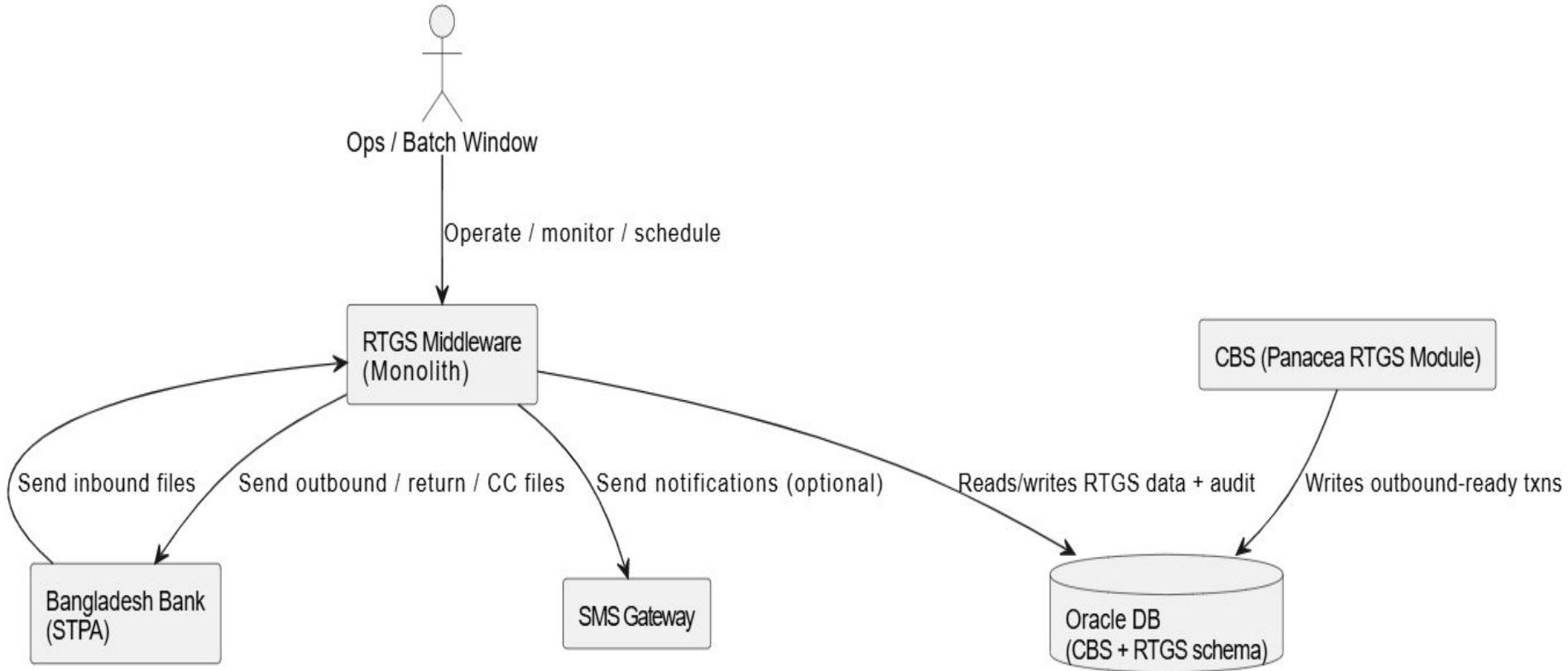
# **RTGS Middleware**

## Now and the future

- Amjad Hossain  
12th April, 2026

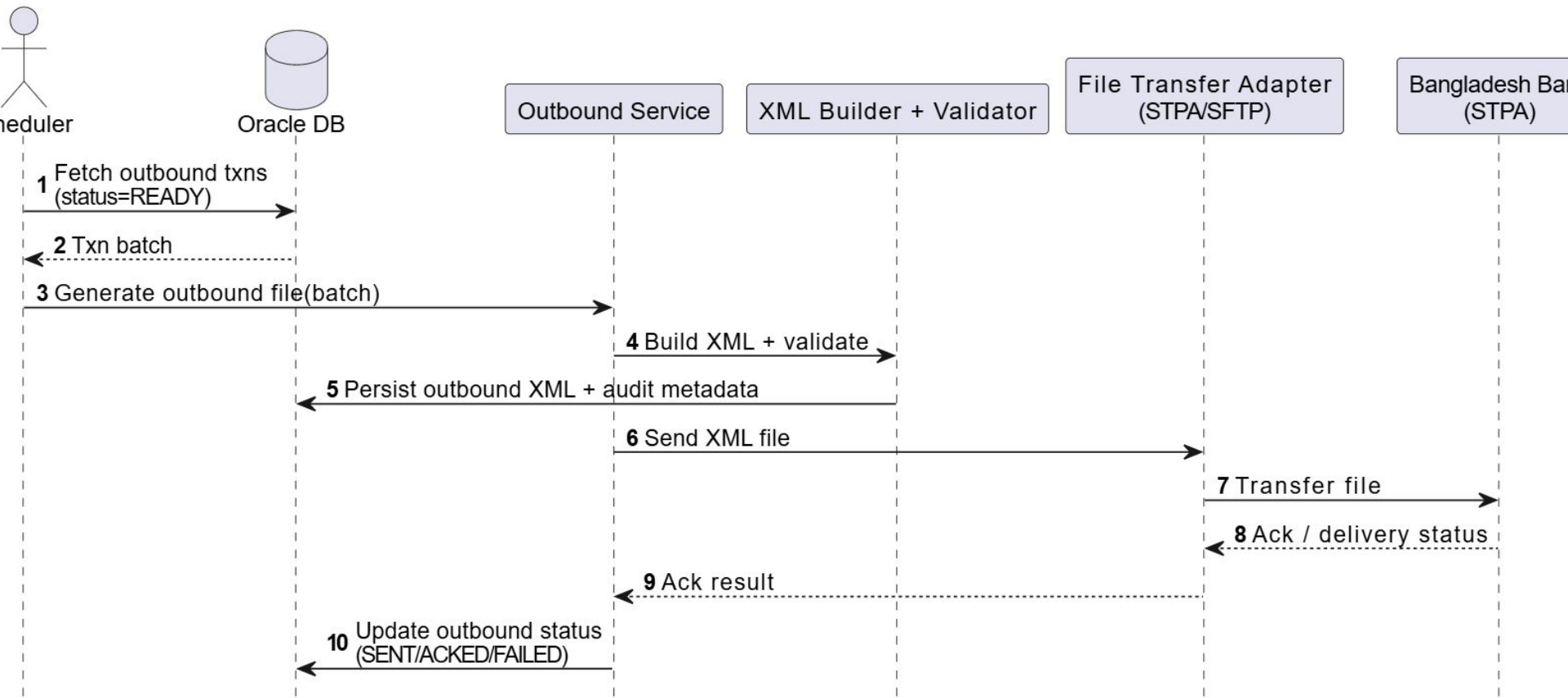
# RTGS Middleware(Monolith): Component diagram

## RTGS Middleware - High-Level Context (Integrations)



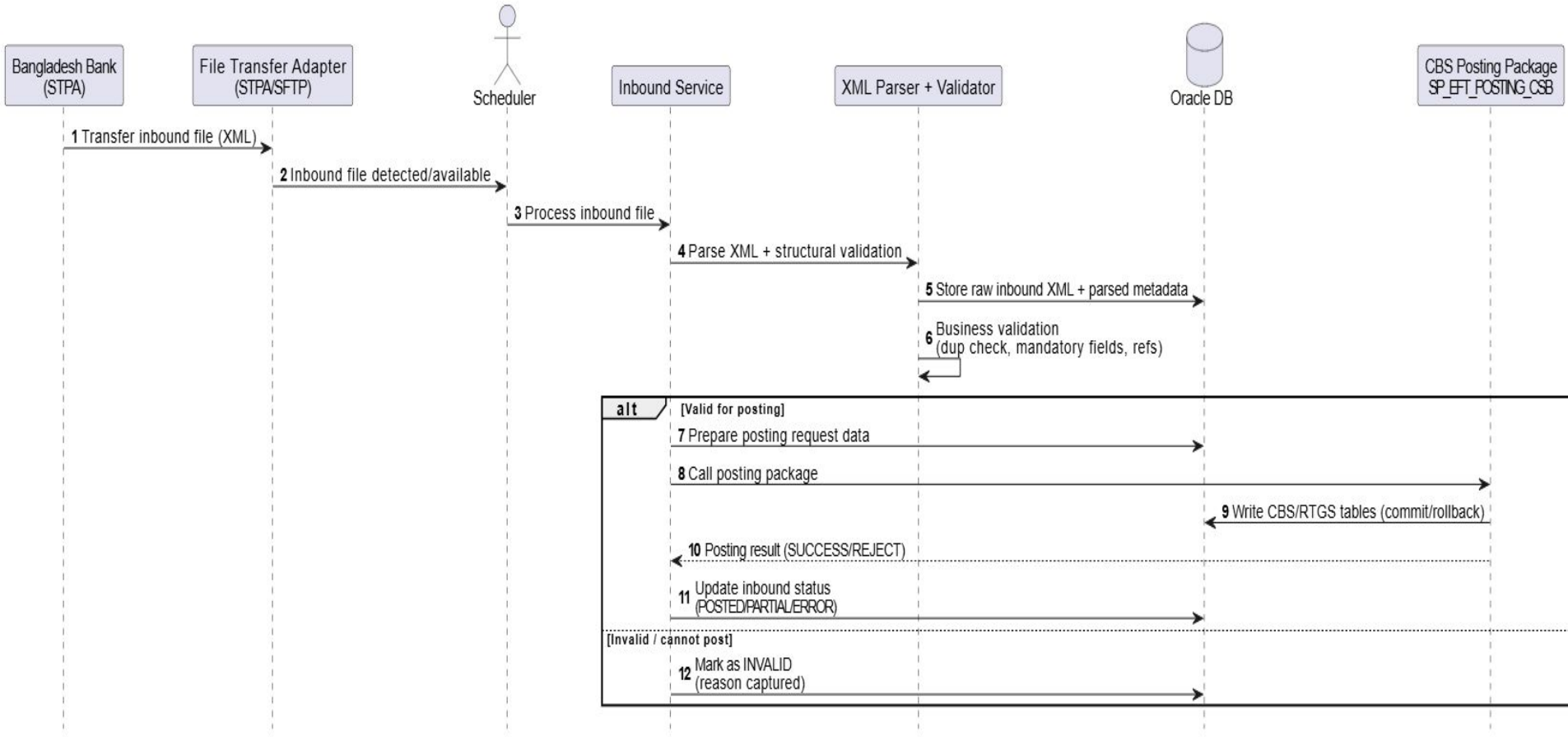
# RTGS Middleware(Monolith): Outbound call

RTGS - Outbound: Build + Send Payment File (CBS -> BB)

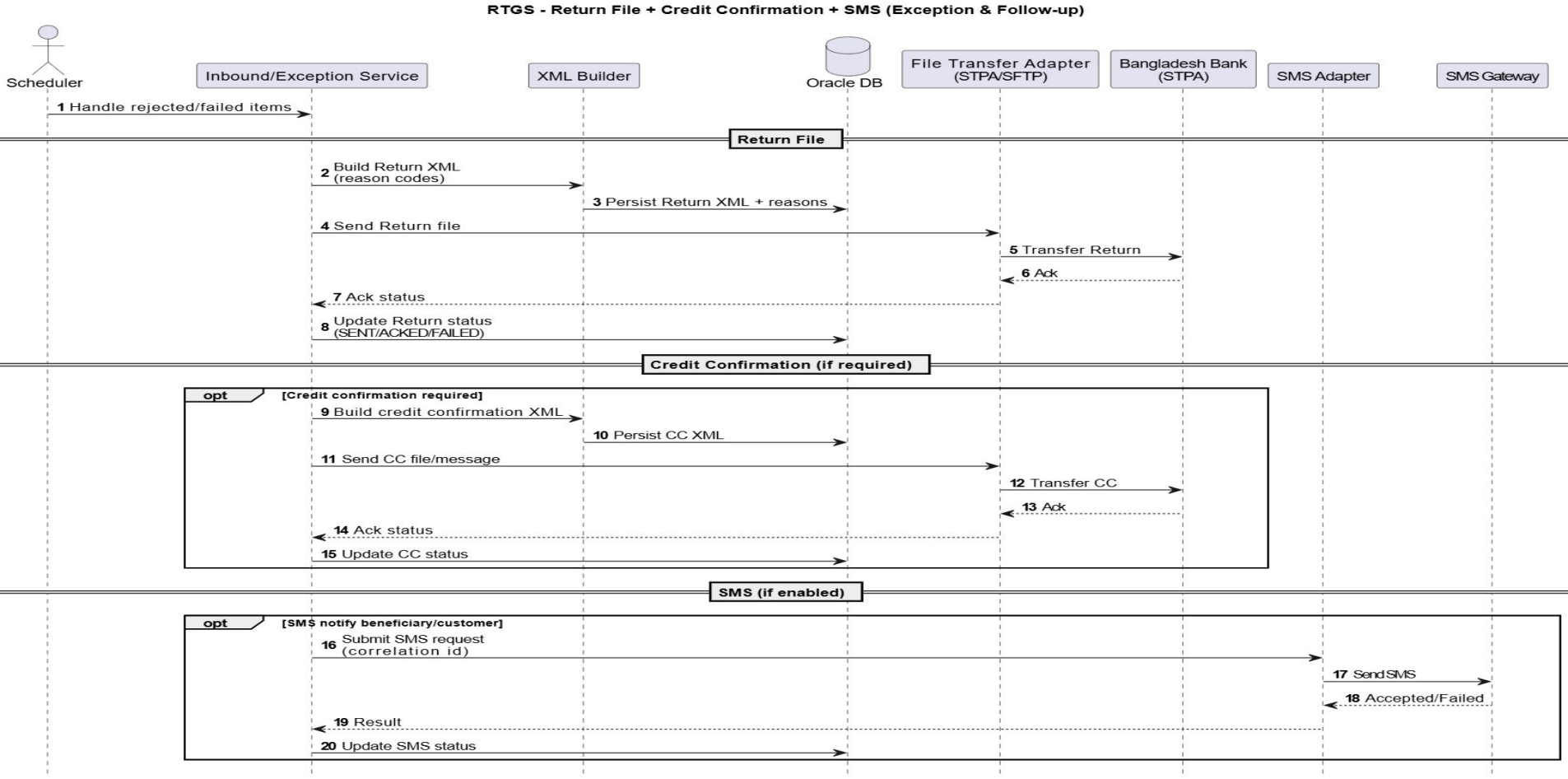


# RTGS Middleware(Monolith): Inbound call

RTGS - Inbound: Receive + Validate + Post to CBS (BB → CBS)



# RTGS Middleware(Monolith): Return and Confirmation



# When microservices makes sense for RTGS

Microservices is justified if you have **at least 2–3** of these in a real, recurring way:

- **Change contention:** multiple teams stepping on each other, releases blocked, “big bang” deployments.
- **Scaling asymmetry:** inbound parsing/validation spikes vs posting vs file transfer; you want to scale parts independently.
- **Reliability isolation:** failures in SMS or file transfer should not impact posting/settlement workflows.
- **High compliance pressure:** you need clearer ownership boundaries, audit trails, and ADRs per capability.
- **Roadmap growth:** RTGS is expanding (new message types, channels, confirmations, more banks/participants, stronger monitoring).

If none of that is true, microservices becomes mostly *extra operational complexity*.

# When microservices makes sense for RTGS

Microservices is justified if you have **at least 2–3** of these in a real, recurring way:

- **Change contention:** multiple teams stepping on each other, releases blocked, “big bang” deployments.
- **Scaling asymmetry:** inbound parsing/validation spikes vs posting vs file transfer; you want to scale parts independently.
- **Reliability isolation:** failures in SMS or file transfer should not impact posting/settlement workflows.
- **High compliance pressure:** you need clearer ownership boundaries, audit trails, and ADRs per capability.
- **Roadmap growth:** RTGS is expanding (new message types, channels, confirmations, more banks/participants, stronger monitoring).

If none of that is true, microservices becomes mostly *extra operational complexity*.

# New Challenges of Microservice in RTGS

RTGS is **stateful, workflow-heavy, and integration-dependent** (CBS package + Oracle + STPA). Microservices brings new challenges here because:

- **Distributed transactions** get messy fast. If posting must be atomic, you'll need **Saga/outbox/idempotency** rigor.
- **Operational burden** increases: service discovery, retries, timeouts, versioning, tracing, on-call playbooks.
- **Data ownership** is hard if you must keep a shared Oracle schema/package for a while.
- **Latency + failure modes** multiply: network hops, partial failures, queue backlogs.

Go with microservices only with a migration pattern: **Strangler**



# Journey to Microservice: **Stabilize the monolith**

Ensure followings before extraction so you don't export chaos:

- Add structured logging with correlation IDs, that help to trace any flow first to last
- Add metrics (TPS, p95 latency per step, queue depth, file transfer time)
- Add idempotency consistently
- Add outbox/inbox tables (even inside monolith) to normalize integration reliability
- Define(written) Test case / Use cases
- Automate tests as much as possible
- Add Unit Tests

# Journey to Microservice: **Modularize**

Refactor into modules with strict boundaries

- Inbound Processing
- Validation/Rules
- Posting Orchestration (CBS package boundary)
- Outbound Generation
- File Transfer Adapter
- XML Parser
- Notifications (SMS)

# Journey to Microservice: **Extract low-risk services**

Start with things that have clear boundaries and don't require shared DB transactions:

1. **Notification Service (SMS)**
2. **File Transfer Service (STPA/SFTP)**
3. **Reporting/Audit Query Service (read-only, via replica)**

These reduce blast radius and simplify the monolith immediately.

# Let's define services

|                                          |                                        |
|------------------------------------------|----------------------------------------|
| 1. RTGS-Ingestion-Service                | 7. RTGS-Outbound-Orchestration-Service |
| 2. RTGS-Canonicalization-Service         | 8. RTGS-Credit-Confirmation-Service    |
| 3. RTGS-Validation-Service               | 9. RTGS-Transfer-Gateway-Service       |
| 4. RTGS-Posting-Orchestrator-Service     | 10. RTGS-Notification-Service          |
| 5. RTGS-CBS-Posting-Facade-Service       | 11. RTGS-Operational-Read-Service      |
| <b>6. RTGS-Return-Management-Service</b> | 12. RTGS-Audit-Ledger-Service          |

# 1. RTGS-Ingestion-Service

## Inbound Intake Service

### Responsibility

- Detect/receive inbound RTGS files from STPA channel (handoff from transfer service)
- Create a canonical “InboundFileReceived” record + immutable raw payload reference
- Basic parsing envelope checks (not full validation)

### Improvements

- Isolates inbound spikes from the rest of the workflow
- Creates a clean ingestion boundary for observability/replay

### Challenges kept/introduced

- Needs strict idempotency for duplicate file deliveries
- Requires durable storage for payload references (DB/object store)

# 2. RTGS-Canonicalization-Service

## RTGS Message Parsing & Canonicalization Service

### Responsibility

- Parse XML into canonical internal message(s)
- Schema validation, structural validation, version handling
- Emit canonical events: `PaymentMessageParsed`, `ParsingFailed`

### Improvements

- Centralizes parsing rules; easier to update BB XML versions
- Removes parsing burden from posting and return generation

### Challenges

- Version drift / XML schema updates become a release driver (but isolated)

# 3. RTGS-Validation-Service

## Validation & Compliance Rules Service

### Responsibility

- Business validation: mandatory fields, duplicates (STAN/reference), allowed values, participant rules
- Decide: **ValidForPosting** vs **RejectedWithReason**

### Improvements

- Makes validation rules independently deployable
- Enables policy-as-code approach later (rules engine if needed)

### Challenges

- Requires consistent reason codes + reason taxonomy (critical for returns)
- Needs fast read-model access for duplicate checks

# 4. RTGS-Posting-Orchestrator-Service

## Posting Orchestrator Service

### Responsibility

- Orchestrate the “post to CBS” workflow for validated items
- Maintain state machine: **READY** → **POSTING** → **POSTED** / **FAILED** / **RETRY**
- Retry with backoff, poison-message handling

### Improvements

- Removes tight coupling between inbound parsing and CBS posting
- Enables controlled throughput to protect CBS/Oracle contention hotspots

### Challenges

- Distributed workflow means you must implement **saga/outbox** discipline
- Exactly-once is not realistic; you need effectively-once via idempotency



# 5. RTGS-CBS-Posting-Facade-Service

## CBS Posting Facade Service

### Responsibility

- The only service allowed to call Oracle package `SP_EFT_POSTING_CSB`
- Input: canonical posting request
- Output: posting result + CBS reference numbers

### Improvements

- Strong boundary around the highest-risk integration
- Makes DB/package coupling explicit and containable

### Challenges

- You still depend on Oracle package behavior and table contention
- Requires careful connection pool sizing, timeouts, circuit breakers

# 6. RTGS-Return-Management-Service

## Return Generation Service

### Responsibility

- For invalid inbound items, generate RTGS “Return” XML with mandated structure
- Track return lifecycle: generated/sent/acked

### Improvements

- Isolates exception flow; avoids polluting posting path
- Clear auditing of rejections (important given current “no auditability” gap)

### Challenges

- Needs robust reason code management and replay-safe generation

# 7. RTGS-Outbound-Orchestration-Service

## Outbound Orchestrator Service

### Responsibility

- Poll CBS/RTGS DB for outward-ready transactions (since outward is generated from CBS)
- Batch, build outbound XML, manage outbound states and acks

### Improvements

- Independent scaling of outbound workload
- Better scheduling control (you have 5 schedulers today)

### Challenges

- Still coupled to CBS RTGS module and the “outward originates in CBS” rule

# 8. RTGS-Credit-Confirmation-Service

## Credit Confirmation Service

### Responsibility

- Generate credit confirmation messages for posted items
- Manage lifecycle and delivery status

### Improvements

- Isolates a downstream flow from core posting
- Enables separate SLAs and retry policies

### Challenges

- Correlation correctness (must map to posting outcomes)
- Backlog handling

# 9. RTGS-Transfer-Gateway-Service

## STPA File Transfer Service

### Responsibility

- Send/receive files to/from Bangladesh Bank via the configured transfer medium
- Delivery guarantees: retries, checksum, ack handling, channel monitoring

### Improvements

- Massive blast-radius reduction: transfer failures don't crash posting
- Standard place to implement network resilience and rate limits

### Challenges

- Operational complexity: credentials, rotation, connectivity, monitoring
- Must implement strict timeouts + backoff to avoid cascading failures

# 10. RTGS-Notification-Service

## Notification (SMS) Service

### Responsibility

- Credit confirmation SMS generation + delivery tracking
- Template, localization, throttling, provider failover

### Improvements

- Isolates external vendor instability
- Scales independently and asynchronously

### Challenges

- Provider delivery semantics, callbacks, billing controls

# 11. RTGS-Operational-Read-Service

## Operational Reporting / Read Model Service

### Responsibility

- Provide operational views (search by STAN/ref, status timelines, error reasons)
- Serves dashboards without touching hot transactional paths

### Improvements

- Removes reporting load from the write DB (currently single RW DB; reporting impacts performance)
- Enables audit-like traceability (currently missing)

### Challenges

- Requires replication/CDC or async projections
- Data consistency is eventually consistent

# 12. RTGS-Audit-Ledger-Service

## Audit & Compliance Ledger Service

### Responsibility

- Immutable audit log of key events: received, validated, posted, returned, sent, acked
- Retention policy enforcement (currently none)

### Improvements

- Solves “no auditability” and “no retention policy” explicitly
- Simplifies investigations and internal audit

### Challenges

- Storage cost, retention rules, privacy constraints



Thank you